

API keys

Musixmatch - API key: bdb46bd4e6b99b9d5a30f96fe5dec81c

Password: hK\$CWFnv0B5pPw29

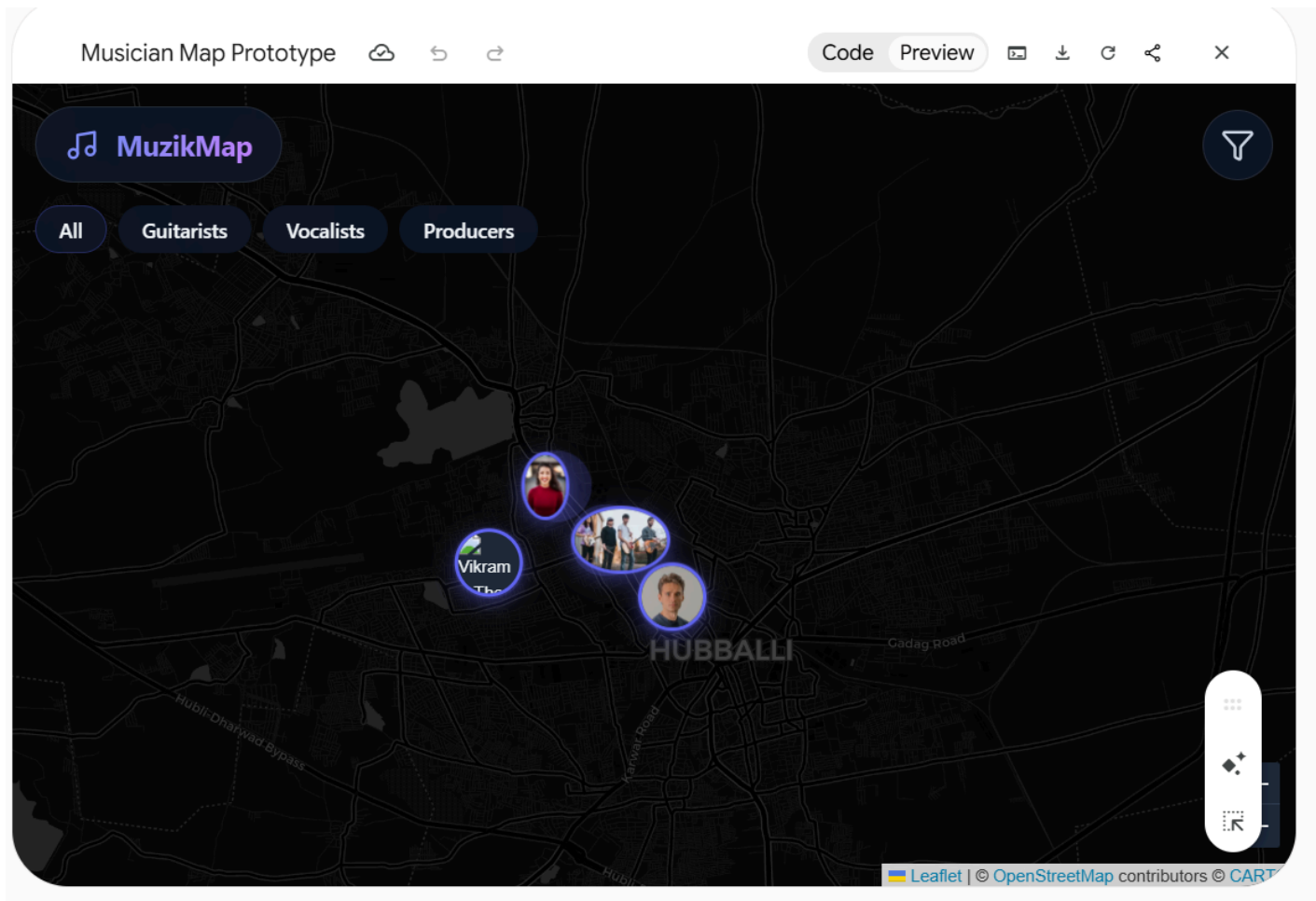
Songstats - c099a6a9-d86f-4520-9ad8-88975de0a6d5

JamBase - API key - jbd_trial_jiX2SJ9wvQoh_mzMHQhPHCmSpvF5CFYNtSaW2914VHECg

LALAL AI - 9cf67ee05f634a8a

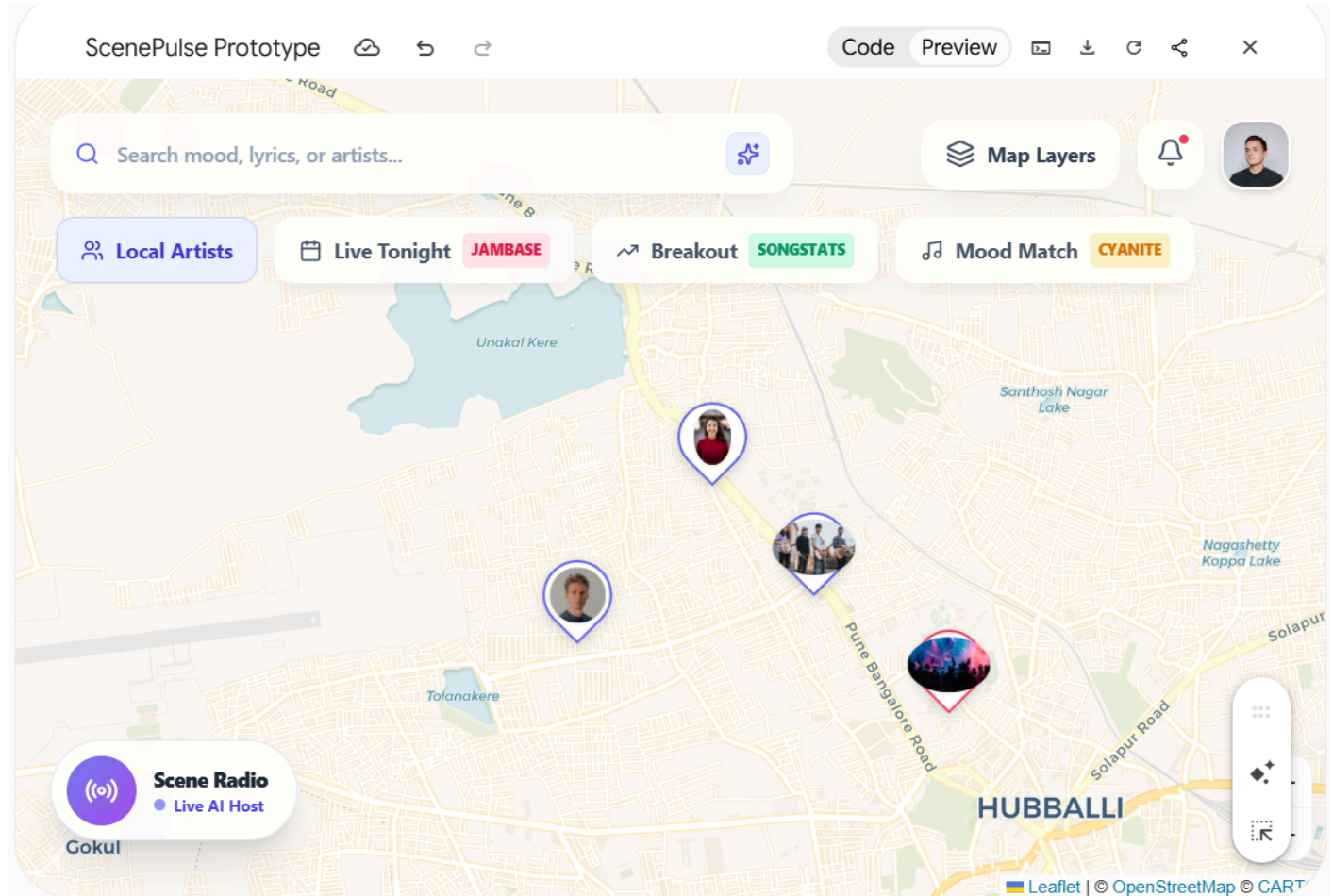
- *Workflow and strategy are below*
- *Attached some snapshots of the prototype below*

First prototype -



Html code - [Untitled document](#)

Second Protoype -



● Writing Lyrics

✕

I was walking down the street



lost inside these neon lights



in the heart of the city

☰ Musixmatch Synced

Ananya S.

Indie Pop Vocalist

SPOTIFY LISTENERS

45.2K ↗

⚙️ CYANITE AUDIO ANALYSIS

MELANCHOLIC

SYNTH POP

VOCALS

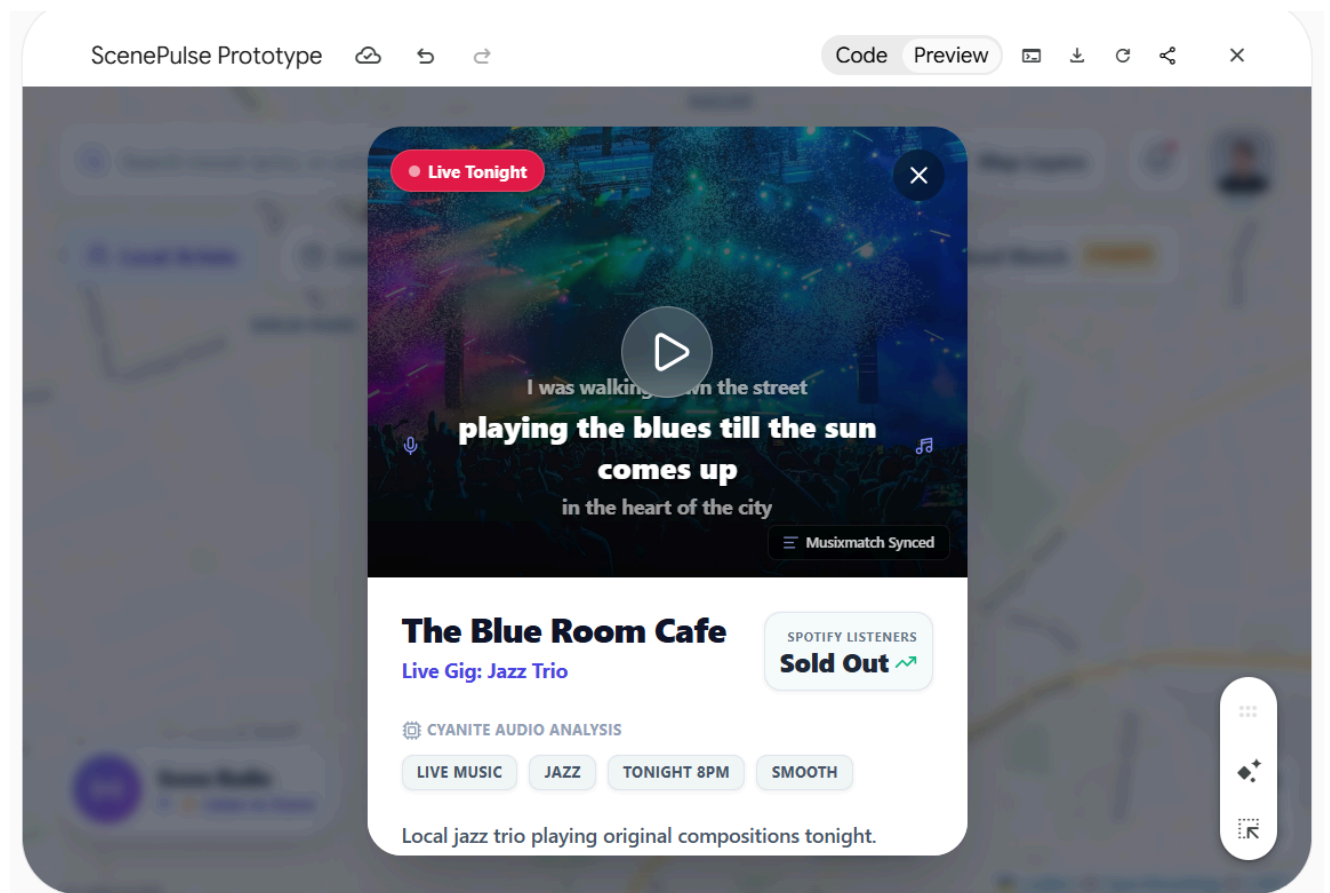
110 BPM

Trained classical singer exploring electronic pop. I write

...

...





Html code - [Untitled document](#)

Strategy -

MuzikMap: "ScenePulse" - The Ultimate Musicathon Pitch

The Core Concept

MuzikMap evolves from a static discovery tool into ScenePulse: a real-time, AI-driven map of a city's local music ecosystem. It connects fans to live gigs, artists to local collaborators, and A&R executives to rising local talent—all powered by deep audio analysis and lyrical metadata.

Instead of just searching by "Guitarist," users can search the map by Mood, Lyric Themes, or Trending Stats.

How to integrate the APIs (The Winning Formula)

1. Musixmatch Pro (The Mandatory Core)

- Semantic Map Search: Allow users to search the map not just by genre, but by Lyrics Analysis. Example: Type "Songs about Heartbreak" or "Summer Vibes" into the map's search bar. The map filters local artists whose original songs match those themes.
- Synced Karaoke Modal: When you click on a local artist's pin, their intro video plays with Musixmatch word-level synced lyrics scrolling on the screen, instantly pulling the user into their music.
- Lyrics Fingerprint: If an artist plays a live gig, fans can use the app to identify the unreleased/local track via lyrics and find the artist's pin on the map.

2. JamBase (The Live Event Layer)

- Gig Pins: Alongside musician avatars, add "Venue" pins. JamBase API populates the map with tonight's local gigs.
- Setlist History: Clicking a gig pin shows the historical setlists of the local band playing there, helping fans decide if they want to go.

3. Songstats (The "A&R Heatmap" Layer)

- Trending Local Filter: Add a heatmap layer to the map. Songstats pulls streaming and TikTok data to show which local artists are currently "blowing up" in your city.
- Profile Metrics: Display real-time Spotify/Apple Music traction inside the artist's dialogue box to establish their credibility to other musicians looking to collaborate.

4. Cyanite (Auto-Tagging & Smart Match)

- Frictionless Onboarding: When a local musician signs up, they just paste their Spotify URL. Cyanite's API analyzes the audio to automatically assign their "Mood," "Energy," and "Sub-genre" tags on the map. No manual data entry required.
- Soundalike Search: A local band needs a new bassist who sounds like the Red Hot Chili Peppers. Cyanite's similarity search finds the exact match in their zip code.

5. Lalal.ai (The "Collab Studio" Tool)

- Stem Separation for Jamming: A vocalist finds a local producer's beat on the map. They use a "Collab Mode" powered by Lalal.ai to extract the instrumental, record a quick vocal over it, and send it directly to the producer's inbox.

6. ElevenLabs (The "AI Scene Radio")

- Local Radio DJ: A floating "Play" button on the map turns on the AI Scene Radio. ElevenLabs' Text-to-Speech creates a highly realistic, hyper-local radio DJ that narrates the map: "What's up Hubballi! Tonight, [Artist Name] is playing at [Venue], and here is their top track..."
- Multilingual Discovery: Translate an artist's bio and have the ElevenLabs voice read it out to tourists in their native language using voice cloning.

7. n8n (The Orchestration Backend)

- Automated Scene Updates: Use n8n to connect it all. Trigger: Musician releases a new song. Action: n8n updates their Cyanite mood tags, fetches Musixmatch lyrics, checks Songstats for playlist placements, and sends an alert to users on the map within a 5-mile radius.

8. Replit (The Infrastructure)

- Speed to Live: Use the Replit Agent to build the Node.js backend that handles the n8n webhooks and serves the map frontend, deploying it instantly to a [.replit.app](https://replit.app) URL for the judges.

The "Wow" Factor for Judges

The judges will love this because it solves a real problem (local music discovery is fragmented) and uses every API synergistically:

1. You find a gig (JamBase).
2. You click the artist playing (Songstats proves they are good).
3. You listen to their song on the map (Cyanite analyzes the mood).
4. You sing along (Musixmatch synced lyrics).
5. The map talks to you (ElevenLabs DJ).

Workflow -

Phase 1: The Skeleton & Map (Hours 1-4)

- **Step 1: Spin up Replit:** Open Replit and create a new `Node.js` project. Use Express for your backend to handle API calls securely (never expose API keys on the frontend).
- **Step 2: Initialize Leaflet:** Create your `index.html`. Add Leaflet.js. Set the starting coordinates to your city (e.g., Hubballi).
- **Step 3: Create the "Pins":** Build the CSS for the custom map pins. For now, create a hardcoded JSON array of 3-4 fake musicians and 1 fake gig to ensure the map renders them correctly.
- **Step 4: The Modal:** Build the HTML/CSS for the popup modal that appears when a pin is clicked. Leave the data fields empty (to be injected by JS later).

Phase 2: The Mandatory Core - Musixmatch (Hours 5-10)

- **Step 1: Backend Route:** In your Replit Node server, create an endpoint: `/api/lyrics/:trackId`.
- **Step 2: Fetch Synced Lyrics:** Use the Musixmatch Pro API to fetch word-level synced lyrics. The response will give you words with timestamps.
- **Step 3: The Karaoke Player:** On the frontend, write a JavaScript function that acts as a "timer" when the user clicks play on an artist's modal. It should read the Musixmatch timestamps and highlight the current lyric line dynamically.
- **Step 4: Semantic Search:** Create a `/api/search` route. When a user types "sad songs", pass this to the Musixmatch *Lyrics Analysis Search* endpoint, get the matching track IDs, and filter your map pins to only show artists who wrote those songs.

Phase 3: Live Events & Data (JamBase & Songstats) (Hours 11-16)

- **Step 1: Bounding Box Query:** Get the current map boundaries (North-East and South-West lat/lng) from Leaflet.
- **Step 2: JamBase API:** Send those coordinates to JamBase. Take the returned gigs, format them into your Pin JSON structure, and render them as Red pins on the map.
- **Step 3: Songstats Integration:** For your featured musicians, make a call to Songstats using their Spotify ID. Grab the "Total Playlist Reach" or "Spotify Listeners" and inject that number into the top right of your frontend modal.

Phase 4: Intelligence & AI (Cyanite, ElevenLabs, n8n) (Hours 17-22)

- **Step 1: Orchestration with n8n:** Set up an n8n workflow. Create a Webhook node.
- **Step 2: Cyanite Auto-Tagging:** When you send a Spotify URL to that n8n webhook, have n8n pass it to Cyanite. Cyanite will return tags like "120 BPM", "High Energy", "Indie Pop". Have n8n send this back to your Replit database to tag the musician.
- **Step 3: ElevenLabs Radio:** Create an `/api/radio` endpoint on your Replit backend. Pass it a dynamic string (e.g., `"Playing tonight at ${gig.venue} is ${gig.artist}..."`). Send this to ElevenLabs TTS API. It will return an audio buffer. Stream that buffer back to the frontend to play when the user clicks the "Scene Radio" button.

Phase 5: The Final Polish (Hours 23-24)

- **Step 1: Error Handling:** APIs will rate-limit or fail. Wrap your frontend fetches in `try/catch` blocks. If Songstats fails, just hide the stats box rather than breaking the whole app.
- **Step 2: The "Happy Path":** Identify the exact sequence of clicks you will do during your presentation. Make sure that specific path loads instantly.
- **Step 3: Demo Recording:** Record your screen clicking through the "Happy Path" just in case the live demo environment has Wi-Fi issues.

